

Datos de fisiología del ejercicio

Laboratorio 1

Mario Morales

Contenido I

- 1 Lectura y preparación de los datos
- 2 Lectura, Limpieza e Imputación en R
- 3 Tratamiento de Datos Faltantes

Section 1

Lectura y preparación de los datos

Objetivo general

Enfrentar al estudiante al “dato sucio” y real. ## Objetivo específico

Realizar la preparación de los datos tomados en el laboratorio para que queden listos para la realización del informe que deberán presentar los estudiantes. La preparación incluye:

- Lectura
- Limpieza
- Imputación
- Anonimización

Section 2

Lectura, Limpieza e Imputación en R

Lectura

Para esta tarea usaremos el sistema R base.

```
library(readxl)
df <- read_excel("DatosLab1.xlsx")
df <- as.data.frame(df)
```

①

②

③

- ① Librería para leer archivos de Excel
- ② Lectura
- ③ Convertimos a `data.frame` puro de R Base para evitar el formato `tibble`

Inspección visual de los datos

```
head(df)
```

	Marca temporal	Dirección de correo electrónico
1	2026-03-11 12:56:49	schavescontreras80@correo.unicordoba.edu
2	2026-03-11 13:38:07	josuedramos@correo.unicordoba.edu
3	2026-03-11 13:52:10	mhlopez@correo.unicordoba.edu
4	2026-03-11 14:31:58	acrivero@correo.unicordoba.edu
5	2026-03-11 14:46:28	lpatriciaramirez@correo.unicordoba.edu
6	2026-03-11 14:52:44	vgaleano@correo.unicordoba.edu

	Nombre Completo	Documento	Fecha de Nacimiento
1	Sofia Chaves Contreras	1.00037298E9	2001-06-
2	Josué David Ramos Sánchez	1.13797762E9	2007-05-
3	Mauro López Martínez	1.062434362E9	2007-02-
4	Andres Camilo Rivero Pacheco	1.067864128E9	2006-09-
5	Laura Patricia Ramírez Polo	1.071.352.721	2007-12-
6	Verónica Galeano Martínez	1.062434986E9	2007-06-

Teléfono / Contacto Consentimiento informado diligenciado

Inspección visual de los datos: Columnas 6 hasta el final

```
head(df[,6:ncol(df)])
```

	Edad	Sexo	Teléfono / Contacto	Consentimiento informado
1	24.0	Femenino	3.106945186E9	
2	18.0	Masculino	3.207847832E9	
3	19.0	Masculino	3.206853212E9	
4	19.0	Masculino	3.21358451E9	
5	18.0	Femenino	3.002720373E9	
6	18.0	Femenino	3.13786016E9	
	Cuestionario PAR-Q diligenciado			Apto para la evaluación / pr
1				Si
2				Si
3				Si
4				Si
5				Si
6				Si

Destrucción de consentimiento de los participantes

Inspección visual de los datos str

```
str(df[,6:ncol(df)])
```

```
'data.frame':  20 obs. of  20 variables:
```

```
$ Edad                                     : chr
$ Sexo                                   : chr
$ Teléfono / Contacto                    : chr
$ Consentimiento informado diligenciado  : chr
$ Cuestionario PAR-Q diligenciado        : chr
$ Apto para la evaluación / práctica    : chr
$ Restricción o condición reportada por el participante: chr
$ Frecuencia cardíaca en reposo (lpm)    : num
$ Presión arterial sistólica (mmHg)      : num
$ Presión arterial diastólica (mmHg)     : num
$ Estatura                              : chr
$ Masa Corporal                         : chr
$ Perímetro de Cintura                  : chr
$ TMC                                   : chr
```

Renombrar columnas

En el Formato de registro de variables, hay nombres largos como Frecuencia cardíaca en reposo. En R base es una buena práctica:

- Evitar espacios: Usar guiones bajos (_).
- Evitar tildes y ñes: Para prevenir problemas de codificación (encoding).
- Minúsculas: R es sensible a mayúsculas (case-sensitive), y escribir todo en minúsculas evita errores comunes de digitación.

Recuperar los nombres actuales

```
colnames(df)
```

```
[1] "Marca temporal"
[2] "Dirección de correo electrónico"
[3] "Nombre Completo"
[4] "Documento"
[5] "Fecha de Nacimiento"
[6] "Edad"
[7] "Sexo"
[8] "Teléfono / Contacto"
[9] "Consentimiento informado diligenciado"
[10] "Cuestionario PAR-Q diligenciado"
[11] "Apto para la evaluación / práctica"
[12] "Restricción o condición reportada por el participante"
[13] "Frecuencia cardíaca en reposo (lpm)"
[14] "Presión arterial sistólica (mmHg)"
[15] "Presión arterial diastólica (mmHg)"
```

Asignar nuevos nombres

Creamos un vector con los nuevos nombres

```
nombres=c("marca_temp","email","nombre","documento","f_nac", ' ')
```

Asignar nuevos nombres

```
colnames(df)=nombres
```

Imprimir los nuevos nombres

```
colnames(df)
```

```
[1] "marca_temp"      "email"           "nombre"          "docum
[5] "f_nac"           "edad"            "sexo"            "tel"
[9] "consentimiento" "par_q"           "apto"            "restr
[13] "fcr"             "presion_sis"     "presión_dia"     "estat
[17] "masa_corporal"   "cintura"         "imc"             "masa_
[21] "masa_muscular"  "masa_osea"       "grasa_vis"       "fuerz
[25] "fuerza_mi"
```

Tipos de datos en las columnas

```
'data.frame':  20 obs. of  14 variables:
 $ edad      : chr  "24.0" "18.0" "19.0" "19.0" ...
 $ fcr       : num  85 75 72 88 74 78 112 78 54 85 ...
 $ presion_sis : num  122 125 113 108 110 105 137 118 124 135 ...
 $ presión_dia : num  90 82 69 71 70 67 33 46 82 64 ...
 $ estatura   : chr  "156.4 cm" "182.0" "166.0" "180.5" ...
 $ masa_corporal: chr  "60.3 kg" "90.7" "54.3" "69.0" ...
 $ cintura    : chr  "73.4 cm" "89.4" "70.5" "73.0" ...
 $ imc        : chr  "24.7 kg/m^2" "27.4" "19.8" "21.2" ...
 $ masa_grasa  : chr  "32.8 %" "25.2" "15.9" "17.8" ...
 $ masa_muscular: chr  "38.4 Kg" "22.9" "43.3" "53.9" ...
 $ masa_osea   : chr  "2.1 kg" "3.4" "2.3" "2.9" ...
 $ grasa_vis   : chr  "3.0" "6.0" "1.0" "2.0" ...
 $ fuerza_md   : chr  "28.0" "32.0" "30.0" "38.0" ...
 $ fuerza_mi   : chr  "26.0" "34.0" "28.0" "40.0" ...
```

Limpieza de “Datos Contaminados” (Uso de Expresiones Regulares)

Para limpiar el “60.3 kg” usaremos `gsub` para eliminar texto y `as.numeric` para transformar.

```
# Limpieza de la columna Edad
# gsub("[^0-9.]", "", x) elimina todo lo que NO sea número o punto
df$edad <- gsub("[^0-9.]", "", df$edad)
df$edad <- as.numeric(df$edad)
# Aplicamos lo mismo a estatura y masa corporal
df$estatura <- as.numeric(gsub("[^0-9.]", "", df$estatura))
df$masa_corporal <- as.numeric(gsub("[^0-9.]", "", df$masa_corporal))
```


Continue usted

Haga lo mismo con las columnas que faltan

```
'data.frame':  20 obs. of  14 variables:
 $ edad      : num  24 18 19 19 18 18 18 20 18 19 ...
 $ fcr       : num  85 75 72 88 74 78 112 78 54 85 ...
 $ presion_sis : num  122 125 113 108 110 105 137 118 124 135 ...
 $ presión_dia : num  90 82 69 71 70 67 33 46 82 64 ...
 $ estatura   : num  156 182 166 180 158 ...
 $ masa_corporal: num  60.3 90.7 54.3 69 52 55.2 67.3 51.5 66 ...
 $ cintura    : num  73.4 89.4 70.5 73 67.8 70 79 68 73.9 82 ...
 $ imc        : num  24.7 27.4 19.8 21.2 21 ...
 $ masa_grasa  : num  32.8 25.2 15.9 17.8 30.6 27.6 19.4 14.4 ...
 $ masa_muscular: num  38.4 22.9 43.3 53.9 34.2 38 51.5 41.8 4 ...
 $ masa_osea   : num  2.1 3.4 2.3 2.9 1.9 2 2.7 2.3 2.6 2.7 ...
 $ grasa_vis   : num  3 6 1 2 1 1 2.5 1 1 5 ...
 $ fuerza_md   : num  28 32 30 38 24 28 40 36 42 42 ...
 $ fuerza_mi   : num  26 34 28 40 24 22 36 34 42 48 ...
```

Anonimización de Datos

Manipularemos las columnas por índice o nombre y a generar identificadores únicos.

```
# 1. Identificar columnas sensibles por nombre
columnas_sensibles <- c("marca_temp", "email", "nombre", "documento")

# 2. Crear un ID aleatorio único para cada fila
set.seed(123)
df$id_estudiante <- sample(1000:9999, nrow(df), replace = FALSE)
```

Anonimización de Datos

Manipularemos las columnas por índice o nombre y a generar identificadores únicos.

```
# 3. Crear el nuevo dataframe omitiendo las sensibles
# Usamos la indexación negativa o selección por nombre
df_anonimo <- subset(df, select= !(names(df) %in% columnas_sensibles))

# 4. Reordenar para que el ID sea la primera columna
df_anonimo <- df_anonimo[, c(ncol(df_anonimo), 1:(ncol(df_anonimo)-1))]
```

Columnas del df anónimo

```
colnames(df_anonimo)
```

```
[1] "id_estudiante"  "edad"           "consentimiento" "par_o
[5] "apto"           "restriccion"    "fcr"             "pres
[9] "presión_dia"    "estatura"       "masa_corporal"  "cintu
[13] "imc"           "masa_grasa"     "masa_muscular"  "masa_
[17] "grasa_vis"     "fuerza_md"      "fuerza_mi"
```

Volver a unir las columnas sensibles

Para eso hay que dejar un data frame con las columnas sensibles y la columna `id_estudiante` que sirve como llave

```
# Creamos un dataframe aparte que SOLO tenga la llave y los no  
df_llave_maestra <- df[, c("id_estudiante", "marca_temp", "email")]
```

Datos desordenados

Simulemos que alguien desordena las filas del data frame anónimo por completo:

```
# Desordenamos las filas aleatoriamente  
df_anonimo_desordenado <- df_anonimo[sample(nrow(df_anonimo)),]
```

Juntemos

Ahora juntamos la información sensible al data después de desordenado

```
# Unimos los nombres de vuelta al archivo desordenado usando  
df_recuperado <- merge(x = df_anonimo_desordenado,  
                        y = df_llave_maestra,  
                        by = "id_estudiante")
```

Custodia de datos

- El archivo que usarán para el informe no tendrá los nombres, solo el `id_estudiante`.
- Si alguno quiere saber de quién es un dato (por ejemplo, quién tiene la mayor fuerza prensil), tendrá que pedírselo al “dueño de la llave”
- Este es el concepto de custodia de datos.

Section 3

Tratamiento de Datos Faltantes

Identificación de Datos Faltantes

- Los estudiantes menores de 18 años, no tienen los datos arrojados por la balanza.
- Primero identificaremos las columnas que tienen datos faltantes, y contaremos cuántos hay

```
# Retorna TRUE si hay al menos un dato faltante en cualquier p  
any(is.na(df_anonimo))
```

```
[1] TRUE
```

```
# Cuenta el total de datos faltantes en toda la matriz  
sum(is.na(df_anonimo))
```

```
[1] 6
```

Identificación por columnas

Es más útil identificar los faltantes por columnas

Podemos usar la función `colSums()` combinada con `is.na()`. Esto nos dará una lista clara de qué variables necesitan atención (como la grasa visceral o la masa ósea en los menores de 18 años).

```
# Retorna el número de NAs por cada columna
conteo_nas <- colSums(is.na(df_anonimo))
print(conteo_nas)
```

id_estudiante	edad	consentimiento	par_q	
0	0	0	0	
restriccion	fcr	presion_sis	presión_dia	
0	0	0	0	
masa_corporal	cintura	imc	masa_grasa	m
0	0	1	2	
masa_osea	grasa_vis	fuerza_md	fuerza_mi	
1	1	0	0	

Qué columnas tienen NA's

La siguiente función nos dice qué columnas tienen datos faltantes

```
cual_na=function(df){
  cual=apply(apply(df,2,is.na),2,any)
  colnames(df)[cual]
}
vars_na=cual_na(df_anonimo)
vars_na
```

```
[1] "imc"           "masa_grasa"    "masa_muscular" "masa_osea"
[5] "grasa_vis"
```

Cuántos NA's hay por columna

```
cols=cual_na(df_anonimo)
apply(apply(df[,cols],2,is.na),2,sum)
```

imc	masa_grasa	masa_muscular	masa_osea	gr
1	2	1	1	

Imputación de faltantes

Luego de identificar las variables de composición con NA's, procedemos a imputar, estimando el dato faltante usando la mediana.